DASS Assignment - 3

Designing and Implementing the IMS System

Deadline: 20th April, 2026

General Logistics

- **Team Size:** The assignment can be done individually or as a team (maximum of 2 students).
- **Submission Platform:** Submission will be done **exclusively via GitHub Classroom**. You may join the GitHub Classroom link from here and create a team with any team name.
- You can find the Doubt Doc here

System Description: Institute Management System (IMS)

The Institute Management System (IMS) is designed to automate the management processes of an educational institute-from student admission to examination management, course scheduling to human resources, and salary management.

Your overarching goal is to model the **entire system** conceptually, but you will only prototype and implement **Dashboard plus end-to-end features of two specific modules** of your choice.

Modules of Institute Management System		
Time Table	Messaging	Attendance
Student Admission	Courses and Batches	Examination
Human Resources	User Management	News Management
Student Details	Finance	Multiple Dashboards

Detailed Capabilities of the IMS Modules

Dashboard

- Iconic dashboard with an innovative 'Search bar' for instant navigation.
- User-friendly interface designed for users with basic computer knowledge.
- Easy to learn with a shallow learning curve; displays latest news on login.
- Language settings and basic configuration (country, currency, time zone).
- General settings: grading systems, automatic unique IDs, etc.
- Manage courses, batches, subjects (including electives), and batch transfers.
- Customizable admission forms and SMS module integration for group/single alerts.
- Manage student categories and graduation facilities.

Admission

- Unique ID generation for all students and comprehensive admission forms.
- Facility to add multiple guardians and emergency contact details.
- Record previous education history and upload student photos.
- Fully customizable to meet specific school standards.

Student Details

- Normal student view based on batches.
- Search functionality for both existing and former students.
- Advanced search with a large number of filters for specific data retrieval.

Examinations

- Create exams based on grades, marks, or custom types; group exams as needed.
- Extensive Report Center: Generate automated, quick, and on-demand reports.
- Statistical and graphical views (charts) for better analytical insights.
- Support for GPA, CCE, and CWA evaluation methods.

Manage Users

- Global search for any user via the dashboard search bar.
- View and edit user profiles, passwords, and specific privileges based on roles.
- Role-based access control to set standards based on organizational responsibility.

Human Resources

- End-to-end employee management from admission to exit.
- Customizable employee admission and payroll forms.
- Robust payroll management with authenticated one-click payslip approval/rejection.
- Efficient leave management system and advanced employee search.

Attendance

- Easy marking of attendance with optional notes/remarks.
- Multiple report types (daily, monthly, subject-wise).
- Filterable attendance reports for quick analysis.

Finance

- Comprehensive fee classification and separate fee collection date design.
- Analysis of fee defaulters and easy fee submission/instant payment processing.
- Track all expenses, incomes, assets, liabilities, and donations.
- Automatic transaction facilities and integration with the payslip system.
- Assign specific tutors to batches for financial tracking.

Messages

- Inbuilt messaging system for administration, teachers, students, and parents.
- Record all communications with students.
- Broadcast information regarding school events, news, and holidays.

Time Table

- Drag-and-drop creation interface for easy scheduling.
- Real-time alerts on subject limits per week and employee workload limits.
- Ability to create, edit, or delete timetables in advance.

Manage News

- Create, edit, and search news using a rich text format (RTF).
- Facility for users to comment on published news.
- Moderation tools to delete comments or news entries.

Task 1: System Design & Report

You must prepare a comprehensive, self-written design report - as a pdf. **Verbosity and unnecessary verbiage will be strictly penalized.** Keep your explanations concise and to the point. The report must contain:

1. **Hand-drawn UML Class Diagram:** Model the *entire* IMS system. Include inheritance (generalization), association, composition relationships, operations, cardinality, and role indicators. You may split it into smaller parts and show relations between them. *Messy and unreadable diagrams will be penalized.*
2. **Responsibilities Table:** A table summarizing the responsibilities of each major class from your UML.
3. **Dynamic Diagrams:**
   - **Sequence Diagrams** for any **3 flows** within the system.
   - **State Diagram** modeling any **1 major entity/part** of the system.
4. **Design Narrative & Analysis:** A brief narrative outlining how your design balances criteria like low coupling, high cohesion, separation of concerns, and the Law of Demeter.
5. **Pattern & Anti-Pattern Analysis:** Discuss at least 2 design patterns and 2 design anti-patterns present (or avoided) in your design.
6. **Reflection:** A short reflection on the two strongest and two weakest aspects of your design.

Feature Selection & Implementation Constraints

While your UML diagrams in Task 1 must model the entire IMS system, your UI prototyping (Task 2) and Android implementation (Task 3) will focus on a specific subset of features.

1. Mandatory Module: The Dashboard

Every team **must** design and implement the Dashboard. *Note:* The extensive list of capabilities provided for the Dashboard in the previous section was for your reference. For Tasks 2 and 3, you have full creative liberty - provided it is intuitive, visually polished, and serves as a functional, end-to-end navigational hub for your app that and includes all essential features.

2. Elective Modules

In addition to the Dashboard, you must select exactly **two** more modules to implement end-to-end. To ensure your prototype covers a diverse range of system interactions and data flows, you must choose **exactly one** module from each of the following two categories:

- **Category A: Core Process Modules (Choose ONE):**

  - Finance
  - Human Resources
  - Examinations
  - Time Table

- **Category B: Data Management Modules (Choose ONE):**
  These modules focus primarily on data entry, retrieval, and user interactions.
  - Student Admission
  - Attendance
  - Student Details *or* Manage Users
  - Messages *or* Manage News

Task 2: UI/UX Prototyping (Figma)

For the selected modules that you plan to implement end-to-end, you need to:

1. **Wireframes:** Build low-fidelity wireframes for your dashboard and the features of the 2 chosen modules using Figma. For those new to wireframes, refer to the link here.
2. **Mock UI (High-Fidelity):** Create high-fidelity mobile-based UI prototypes for these features in Figma. The design must be visually polished and well desgined.
3. **Design Inspiration:** Explicitly mention **3 mobile apps/websites** you took inspiration from. It must be evident in your report and design what UI/UX aspects were taken from which sources.
4. Your UI must follow atleast 3 usability principles from the slides which must be evident **throughout the app.**

Task 3: Android Implementation

You are required to build a functional Android app prototype for the **dashboard and chosen modules** designed in Task 2. **Make sure in your `build.gradle` you have a `BuildConfigField` of String type with the exact name `APP_IDENTIFIER` to be set to the value `your_rollnumber.teammates_rollnumber` (or `your_rollnumber.0` if doing the assignment individually)**

1. **Tech Stack Restriction:** The UI **must** be built natively using **Jetpack Compose**. Frameworks like Flutter, React Native, etc., are strictly prohibited. *(Students without Android devices must use the Android Studio Emulator).*
2. **End-to-End Functionality:** The selected modules must work seamlessly from end to end within the prototype. The UI should match your high-fidelity Figma designs closely. It is also important that your prototype **must** follow the same workflow i.e support the exact same set of features as in our actual IMS system.
3. **Codebase Documentation:** Provide documentation for your codebase. This must be **short, concise, and to the point**. Unnecessary verbosity in documentation will attract penalties.

Note: *As you do not have access to a server for data, you may store the database in the app itself and simulate the end-to-end functionality by having stubs and easy role switching.*

Strict Rules & Guidelines

- **Zero AI Tolerance:** The use of LLMs (ChatGPT, Claude, etc.) or any AI tools for generating the report, documentation, or code is **strictly not allowed**. Any detected AI usage will result in severe penalties.
- **Self-Written Content:** All reports, narratives, and documentations must be entirely self-written.
- **No Verbosity:** Reports and documentation must be dense with value. Fluff, filler text, and verbiage will lead to negative marking.

Grading Rubric (Approximate)

Please note that while deliverables are graded, **the Viva will carry the majority weight** to verify your understanding, design decisions, and code authenticity.

Component	Description	Weightage
Task 1: System Design	Hand-drawn UML (entire system), 3 Sequence diagrams, 1 State diagram, and Design/Pattern Analysis (Concise report).	15%
Task 2: UI/UX Prototype	Low-fi wireframes, High-fi Figma mockups for the features, application of 3 usability principles, and 3 design inspirations.	10%
Task 3: Implementation	Jetpack Compose Android app executing the selected features end-to-end. Concise code documentation.	25%
Viva & Defense	**Primary evaluation component.** Defense of design choices, codebase understanding, UI flow explanation, and system knowledge.	**50%**

Note: *Deductions will be strictly applied for unauthorized tech stacks, messy diagrams, AI usage, and verbose documentation - in which case straight 0 in the implementation section too.*

Submission Instructions

Everything must be pushed onto Github classroom repository with all links included. The codebase must be well organized and clean. DO NOT push build or generated files.